



Software Factory en la Era de la IA

Fundamentos

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Que Es Software Factory

¿Qué es una Software Factory?

Una **Software Factory** es un modelo organizacional que aplica principios de manufactura al desarrollo de software, buscando consistencia, escalabilidad y calidad en la entrega de soluciones digitales.

Definición

Una Software Factory es un entorno estructurado donde el desarrollo de software se gestiona como un proceso productivo, con estándares, roles definidos, herramientas compartidas y métricas de desempeño.

A diferencia del desarrollo tradicional ad-hoc, la Software Factory establece:

- **Procesos estandarizados** para cada fase del ciclo de vida
- **Roles y responsabilidades** claramente definidos
- **Herramientas comunes** compartidas entre equipos
- **Métricas objetivas** para medir productividad y calidad

Características principales

Característica	Descripción
Estandarización	Metodologías, plantillas y patrones reutilizables
Medición	Métricas de productividad, calidad y tiempo
Automatización	CI/CD, testing automatizado, infraestructura como código
Mejora continua	Retrospectivas, análisis de causa raíz
Escalabilidad	Capacidad de crecer sin perder calidad
Gobernanza	Auditorías, revisiones de código, compliance

Componentes de una Software Factory

SOFTWARE FACTORY		
PERSONAS	PROCESOS	HERRAMIENTAS
Roles	Metodos	IDE, repos, CI/CD
Skills	Flujos	Testing, monitoring
Cultura	Gates	PLM, tracking

Personas

- **Roles definidos:** Tech Lead, Architect, Developer, QA, DevOps
- **Cultura de ownership:** cada equipo es responsable de su deliverable
- **Upskilling continuo:** capacitación en nuevas tecnologías y prácticas

Procesos

- **SDLC documentado:** desde intake hasta deployment
- **Gates de calidad:** code review, testing, security scan
- **Definición de Done (DoD):** criterios claros de completitud

Herramientas

- **Repositorios centralizados:** Git, branching strategies
- **CI/CD pipelines:** automatización de build, test y deploy
- **Tracking:** Jira, Azure DevOps, o equivalentes

Diferencias: Software Factory vs Desarrollo Tradicional

Aspecto	Desarrollo Tradicional	Software Factory
Proceso	Ad-hoc, por proyecto	Estandarizado, repetible
Conocimiento	En individuos	En la organización
Calidad	Depende del equipo	Garantizada por proceso
Escalabilidad	Difícil	Diseñada para escalar
Medición	Subjetiva	Objetiva y continua
Onboarding	Largo, depende de mentor	Guiado por playbooks

Componentes típicos

1. **Knowledge Management:** documentación viviente, no specs muertas
2. **Reusable Assets:** librerías, frameworks internos, templates
3. **Quality Gates:** revisiones automáticas y manuales
4. **Metrics Dashboard:** velocity, defect density, lead time
5. **Training Programs:** onboarding estructurado

Software Factory en la Era de la IA

Con la llegada de los LLMs y coding assistants, el concepto de Software Factory evoluciona:

- **Automatización aumentada:** agents que generan código, tests y docs
- **Decisiones asistidas:** AI para code review y architecture suggestions
- **Velocidad multiplicada:** ciclos más cortos, más iteraciones
- **Nuevos roles:** AI Engineer, Prompt Engineer, Agent Orchestrator

Relación con otros conceptos

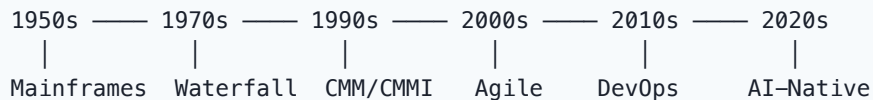
- Se sustenta en principios de ingeniería sólidos
- Se mide con métricas de productividad
- Se gestiona mediante procesos y flujos definidos
- Se complementa con prácticas DevOps

Evolucion Software Factory

Evolución de la Software Factory

La transformación de talleres artesanales a fábricas de software productivas sigue una trayectoria de más de 70 años.

Línea de tiempo



1950s–1960s: La era de los Mainframes

- Programadores como artesanos individuales
- Código en lenguajes de máquina y ensamblador
- **No existía** el concepto de proceso de ingeniería
- Cada sistema era único y artesanal

Características clave:

- Desarrollo sin metodología formal
- Documentación mínima
- Dependencia total del programador

1970s: Waterfall y los primeros procesos

Winston Royce (1970) formalizó el modelo de cascada:

Requisitos → Diseño → Implementación → Testing → Mantenimiento

Logros:

- Primera formalización del proceso de software
- Separación de fases
- Documentación como entregable

Limitaciones:

- Rigidez ante cambios
- Feedback tardío (el cliente ve el producto al final)
- Integración como bottleneck

1980s–1990s: CMM y la medición de madurez

El CMMI (Capability Maturity Model) introdujo:

Nivel	Nombre	Descripción
1	Inicial	Procesos ad-hoc
2	Repetible	Gestión de proyectos básica
3	Definido	Procesos estandarizados
4	Gestionado	Métricas y control cuantitativo
5	Optimizado	Mejora continua

Impacto:

- Las organizaciones empezaron a medir su madurez
- Surgieron las primeras "fábricas de software" en India y Latinoamérica
- Outsourcing y nearshoring crecieron

2000s: Agile rompe el molde

El Manifiesto Ágil (2001) cambió las reglas:

Valores fundacionales:

- Individuos y interacciones > procesos y herramientas
- Software funcionando > documentación extensiva
- Colaboración con cliente > negociación de contratos
- Responder al cambio > seguir un plan

Frameworks que emergieron:

- Scrum (1995, formalizado 2001)
- XP (Extreme Programming, 1999)
- Kanban (adaptado para software, 2000s)

2010s: DevOps y la automatización

DevOps unificó desarrollo y operaciones:

- **CI/CD**: integración y despliegue continuo
- **Infraestructura como Código** (IaC): Terraform, Ansible
- **Containerización**: Docker, Kubernetes
- **Monitoring**: observabilidad como práctica estándar

Resultado: time-to-market reducido de meses a horas.

2020s: AI-Native y la nueva frontera

La Era de la IA redefine la factory:

- **Coding assistants**: GitHub Copilot, Claude, GPT
- **AI agents**: generación autónoma de código y tests
- **Automatización de decisiones**: architecture suggestions, code review

- **Prompt engineering** como skill fundamental

Era	Paradigma	RoI del humano
1950s	Artesanal	Programador único
1970s	Procesado	Ingeniero de proceso
1990s	Medido	Gerente de calidad
2000s	Ágil	Colaborador
2010s	Automatizado	Orquestador
2020s	AI-Native	Estratega y supervisor

Lecciones de la evolución

1. **Los procesos maduran**, pero no deben volverse burocracia
2. **La automatización** siempre acelera, pero requiere gobernanza
3. **La cultura** importa tanto como las herramientas
4. **La medición** es necesaria pero no suficiente
5. **La adaptabilidad** es la única constante

Relación con la SF moderna

La Software Factory actual integra todas estas etapas:

- Procesos del CMMI para gobernanza
- Agilidad para iteración
- DevOps para automatización
- IA para multiplicar capacidades
- Lean para eliminar desperdicio

Era la Contexto

La Era de la IA: Contexto y Cambio de Paradigma

Estamos viviendo una transformación sin precedentes en la forma en que se construye software. Los Large Language Models (LLMs) y los AI agents están redefiniendo el rol del ingeniero de software.

¿Qué cambió?

Antes de la IA generativa

- El programador escribía **todo** el código
- La documentación era escrita **después** (o nunca)
- Las decisiones de arquitectura dependían **exclusivamente** de experiencia humana
- El debugging era **100% manual**

Después de la IA generativa

- Los LLMs **generan código** funcional a partir de prompts
- Los AI agents **ejecutan tareas** de forma autónoma
- La documentación se genera **junto con** el código
- Las decisiones se **asisten** con datos y sugerencias

Componentes del nuevo ecosistema

Componente	Ejemplo	Función
LLMs	GPT-4, Claude, Gemini	Generación de texto y código
Coding Assistants	Copilot, Cursor, Cline	Sugerencias en tiempo real
AI Agents	Devin, OpenCode, AutoGPT	Tareas autónomas end-to-end
RAG Systems	Context7, MCP servers	Contexto específico del dominio
Prompt Engineering	—	Comunicación efectiva con IA

¿Qué significa para los desarrolladores?

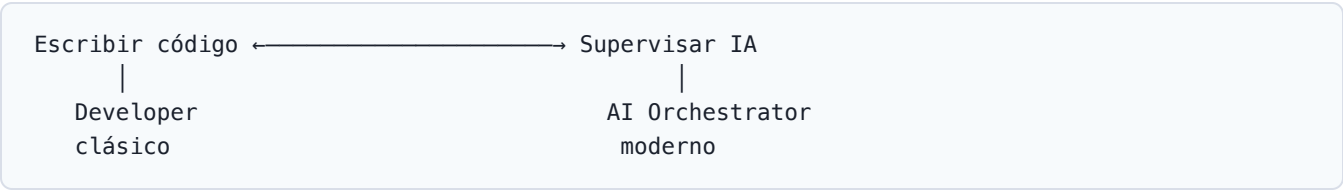
Lo que NO cambia

- La **necesidad de arquitectura** sólida
- La importancia de principios de ingeniería
- La necesidad de **testing** y validación
- El **ownership** del código producido
- La deuda técnica sigue existiendo

Lo que SÍ cambia

- **Velocidad de prototipado:** de horas a minutos
- **Curva de aprendizaje:** nuevos lenguajes y frameworks más accesibles
- **Rol del desarrollador:** de "escribir código" a "orquestrar soluciones"
- **Focus:** más en diseño, menos en sintaxis

El nuevo spectrum de roles



Habilidades en ascenso

1. **Prompt Engineering:** comunicar intención claramente
2. **Code Review asistido:** validar output de IA
3. **Architecture Design:** decidir QUÉ construir
4. **System Thinking:** entender efectos en cascada
5. **Domain Expertise:** conocimiento del negocio

Habilidades que se automatizan

- Escritura de boilerplate code
- Testing repetitivo
- Documentación estándar
- Refactorizaciones mecánicas
- Búsqueda de snippets

Impacto en la Software Factory

La Software Factory se transforma:

Aspecto	Antes	Ahora
Productividad	Horas por feature	Minutos por feature
Calidad	Code review manual	AI-assisted review
Documentación	Escrita después	Generada en paralelo
Testing	Test cases manuales	Auto-generados
Onboarding	Semanas	Días con AI guides

Riesgos y consideraciones

- **Alucinaciones:** la IA puede generar código incorrecto o inseguro
- **Dependencia:** perder skills fundamentales
- **Deuda técnica:** generar código rápido sin diseñar
- **Seguridad:** código con vulnerabilidades no detectadas
- **Privacidad:** datos enviados a servicios externos

Estrategia recomendada

1. **Aprende los fundamentos:** no los saltés por usar IA
2. **Usa IA como acelerador,** no como reemplazo
3. **Revisa siempre** el código generado
4. **Mide impacto:** velocity, calidad, satisfacción
5. **Mantén la curiosidad:** la tecnología cambia rápido

Relación con otros conceptos

- Fundamenta la evolución hacia plataformas de ingeniería
- Complementa DevOps con automatización inteligente
- Requiere segmentación de software más fina
- Impacta directamente en métricas de productividad

Principios Ingenieria Software

Principios de Ingeniería de Software

Los principios de ingeniería son guías fundamentales que ayudan a construir software mantenible, escalable y robusto. Son la base sobre la que se sostiene cualquier Software Factory.

Principios SOLID

Los principios SOLID fueron introducidos por Robert C. Martin y son la piedra angular de la Arquitectura Limpia.

Principio	Significado	Ejemplo
S - Single Responsibility	Una clase, un motivo para cambiar	UserAuth y UserLogger separados
O - Open/Closed	Abierto a extensión, cerrado a modificación	Strategy pattern para pagos
L - Liskov Substitution	Subtipos deben ser sustituibles	PayPalProcessor reemplaza CardProcessor
I - Interface Segregation	Interfaces pequeñas y específicas	No forzar métodos no usados
D - Dependency Inversion	Depender de abstractions, no concreciones	Injectar IEmailService

Ejemplo práctico de SOLID

```
# ❌ Viola SRP - una clase hace todo
class UserService:
    def create_user(self, data): ...
    def send_welcome_email(self, user): ...
    def log_activity(self, action): ...

# ✅ Cumple SRP - responsabilidades separadas
class UserCreator:
    def create(self, data): ...

class WelcomeEmailSender:
    def send(self, user): ...

class ActivityLogger:
    def log(self, action): ...
```

DRY — Don't Repeat Yourself

"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system." — Andy Hunt & Dave Thomas

Aplicación:

- Funciones reutilizables en lugar de código duplicado
- Configuración centralizada
- Templates y componentes compartidos
- Libraries internas en la Software Factory

Cuidado: DRY no significa copiar dos veces = refactorizar. A veces la duplicación intencional es mejor que una abstracción prematura.

KISS — Keep It Simple, Stupid

"Make everything as simple as possible, but not simpler." — Einstein

Aplicación:

- Preferir soluciones simples sobre ingeniosas
- Evitar over-engineering
- Código legible > código clever
- Documentación clara y concisa

YAGNI — You Aren't Gonna Need It

"Always implement things when you actually need them, never when you just foresee that you need it."

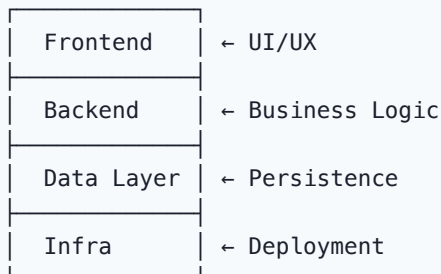
Aplicación:

- No construir features "por si acaso"
- Refactorizar cuando la necesidad es real
- MVP primero, optimizar después
- Complementa el mindset de Lean Software

Separation of Concerns (SoC)

"Separate things that serve different purposes."

En la práctica:



Cada capa se modifica independientemente. Fundamento de la Arquitectura Limpia.

Law of Demeter (LoD)

"Only talk to your immediate friends."

Regla: Un módulo no debe conocer los detalles internos de los objetos con los que interactúa.

```
# ❌ Viola LoD
order.customer.address.city

# ✅ Cumple LoD
order.get_shipping_city()
```

Resumen de principios

Principio	Enfoque	Pregunta clave
SOLID	Diseño de clases	¿Es esta clase cohesiva?
DRY	Reutilización	¿Estoy repitiendo conocimiento?
KISS	Simplicidad	¿Hay una forma más simple?
YAGNI	Necesidad	¿Realmente necesito esto ahora?
SoC	Separación	¿Estas cosas cambian juntas?
LoD	Acoplamiento	¿Estoy hablando con amigos de amigos?

Aplicación en la práctica

Estos principios no son reglas absolutas sino guías. Su aplicación depende del contexto:

- En prototipos rápidos, KISS y YAGNI prevalecen
- En sistemas críticos, SOLID y SoC son innegociables
- En equipos grandes, DRY y LoD reducen fricción
- En deuda técnica, estos principios son la brújula

Relación con otros conceptos

- Base de la Arquitectura Limpia
- Guía para prácticas de desarrollo
- Fundamento de Agile
- Aplicables en desarrollo asistido por IA

Modelos Madurez

Modelos de Madurez: CMMI, SPICE, ISO/IEC 33000

Los modelos de madurez permiten a las organizaciones evaluar y mejorar sus procesos de ingeniería de software de forma estructurada.

¿Qué es un modelo de madurez?

Un modelo de madurez define un conjunto de **prácticas esperadas** para cada nivel de capacidad organizacional. Permite:

- **Evaluar** el estado actual de procesos
- **Planificar** mejoras incrementales
- **Compararse** con estándares de la industria
- **Certificarse** ante clientes y socios

CMMI — Capability Maturity Model Integration

Desarrollado por el Software Engineering Institute (SEI) de Carnegie Mellon.

Niveles de madurez

Nivel	Nombre	Descripción	Prácticas clave
1	Inicial	Procesos impredecibles	Heroísmo individual
2	Gestionado	Proyectos gestionados	Planificación, seguimiento
3	Definido	Procesos estandarizados	Organización, capacitación
4	Cuantitativamente Gestionado	Métricas y control	Análisis estadístico
5	Optimizado	Mejora continua	Innovación, optimización

Áreas de proceso (examples)

Gestión de Proyectos:

- Planificación de proyectos (PP)
- Seguimiento y control (PMC)
- Gestión de riesgos (RSKM)

Ingeniería:

- Desarrollo de requerimientos (RD)
- Solución técnica (TS)
- Integración del producto (PI)

Soporte:

- Gestión de configuración (CM)
- Aseguramiento de calidad (PPQA)
- Análisis y resolución (AR)

Pros y contras del CMMI

Pros	Contras
Framework completo y probado	Costoso de implementar
Reconocimiento internacional	Enfoque en documentación
Enfoque en procesos	Puede generar burocracia
Aplicable a múltiples dominios	Certificación expensive

SPICE — Software Process Improvement

ISO/IEC 33000 es la estandarización internacional de SPICE (Software Process Improvement and Capability dEtermination).

Estructura de process assessment

Dimensión	Descripción
Procesos	Qué se hace (process reference model)
Capacidad	Qué tan bien se hace (niveles 0-5)
Resultados	Qué evidencia existe (process outcomes)

Niveles de capacidad SPICE

Nivel	Nombre
0	Proceso incompleto
1	Proceso realizado
2	Proceso gestionado
3	Proceso establecido
4	Proceso predecible
5	Proceso en optimización

SPICE vs CMMI

Aspecto	SPICE	CMMI
Enfoque	Process assessment	Process improvement
Uso	Evaluación objetiva	Guiar mejora
Flexibilidad	Más modular	Más integral
Adopción	Europa, Asia	Global

ISO/IEC 33000

La familia ISO 33000 reemplaza y armoniza los modelos existentes:

- **ISO/IEC 33001**: Concepts and terminology
- **ISO/IEC 33002**: Requirements for process assessment
- **ISO/IEC 33004**: Requirements for process reference models
- **ISO/IEC 33020**: Process measurement framework

Comparación de modelos

Característica	CMMI	SPICE/ISO 33000	ISO 9001
Dominio	Software/Services	Software/Services	General
Enfoque	Madurez + Capacidad	Capacidad	Sistema QA
Certificación	SCAMPI Appraisal	Formal Assessment	Auditoría
Coste	Alto	Medio	Medio
Actualización	CMMI v2.0 (2018)	ISO 33000 (2015)	ISO 9001:2015

¿Cuándo usar cada modelo?

- **CMMI**: Organizaciones grandes que buscan transformación de procesos
- **SPICE/ISO 33000**: Evaluación objetiva para comparar capacidades
- **ISO 9001**: Complemento para gestión de calidad general
- **Ninguno**: Startups y equipos pequeños pueden beneficiarse de prácticas ágiles sin formalización pesada

Relación con la Software Factory

Los modelos de madurez son especialmente relevantes para la Software Factory porque:

1. Establecen **baseline** de procesos
2. Habilitan **medición** objetiva
3. Facilitan **mejora continua**
4. Soportan **gobernanza** organizacional
5. Complementan Agile con estructura

En la era de la IA

Con la Era de la IA, los modelos de madurez evolucionan:

- **AI-ready processes**: procesos preparados para automatización
- **AI-assisted assessment**: evaluaciones asistidas por IA
- **Nuevo nivel 6** (propuesto): AI-augmented processes

Manifiesto Agile

El Manifiesto Ágil y sus 12 Principios

En febrero de 2001, 17 profesionales de software se reunieron en Snowbird, Utah y crearon el **Agile Manifesto**. Este documento fundamental cambió la forma en que el mundo construye software.

Los 4 valores del Manifiesto

Estamos descubriendo mejores formas de desarrollar software haciendo primero y luego ayudando a otros a hacerlo.

Valor izquierdo (valoramos más)	Valor derecho (pero también valoramos)
Individuos e interacciones sobre procesos y herramientas	Procesos y herramientas
Software funcionando sobre documentación extensiva	Documentación extensiva
Colaboración con el cliente sobre negociación de contratos	Negociación de contratos
Respuesta ante el cambio sobre seguir un plan	Seguir un plan

Nota: Los valores de la derecha tienen valor, pero valoramos más los de la izquierda.

Los 12 Principios

Prioridad al cliente

- 1. Nuestra máxima prioridad es satisfacer al cliente mediante la entrega temprana y continua de software valioso.
- 1. Los cambios en los requerimientos se bienvenidos, incluso tarde en el desarrollo. Los procesos ágiles aprovechan el cambio para ventaja competitiva del cliente.
- 1. Entregar software funcionando frecuentemente, desde un par de semanas a un par de meses, con preferencia a un plazo más corto.

Construir equipos

- 1. Las personas que interactúan con el negocio deben trabajar juntamente con los ingenieros de software durante todo el proyecto.
- 1. Construir proyectos alrededor de individuos motivados. Darles el entorno y soporte necesarios, y confiar en que harán el trabajo.
- 1. La comunicación cara a cara es el método más eficiente y efectivo dentro de un equipo.

Calidad técnica

- 1. El software funcionando es la medida principal de progreso.

- 1. Los procesos ágiles promueven un desarrollo sostenible. Los patrocinadores, desarrolladores y usuarios deben ser capaces de mantener un ritmo constante indefinidamente.
- 1. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.

Mejora continua

- 1. La simplicidad — el arte de maximizar la cantidad de trabajo no realizado — es esencial.
- 1. Las mejores arquitecturas, requerimientos y diseños emergen de equipos auto-organizados.
- 1. A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo y ajusta su comportamiento en consecuencia.

Aplicabilidad hoy

¿Sigue vigente?

Sí, pero con matices:

Principio	Relevancia 2026	Adaptación
Customer collaboration	Alta	Integrar AI como canal
Working software	Alta	CI/CD como estándar
Respond to change	Crítica	AI para pivotar rápido
Individuals & interactions	Alta	Remote-first, async

Frameworks ágiles actuales

- **Scrum**: iteraciones de 1-4 semanas
- **Kanban**: flujo continuo, WIP limits
- **SAFe**: escalado para grandes organizaciones
- **Shape Up**: ciclos de 6 semanas
- **Trunk-based**: desarrollo continuo en main

Errores comunes de implementación

- 1. **"Agile" = Scrum**: Agile es una filosofía, no un framework
- 2. **Sin retrospectivas**: pierdes la mejora continua
- 3. **Over-engineering**: viola el principio de simplicidad
- 4. **Sin definición de Done**: calidad inconsistente
- 5. **Micro-management**: contradice auto-organización

Agile + DevOps + IA

La combinación de estos tres pilares crea la Software Factory moderna:



- **Agile** provee el mindset y los valores
- DevOps provee la automatización
- La IA provee la aceleración

Relación con otros conceptos

- Complementa Lean Software Development
- Se integra con DevOps
- Se soporta en principios de ingeniería
- Se mide con métricas ágiles

Devops Filosofia

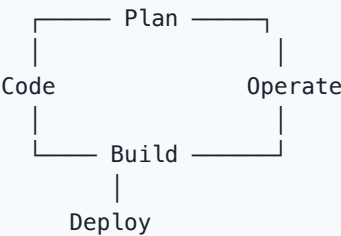
DevOps como Filosofía

DevOps no es solo herramientas o pipelines. Es una **filosofía cultural y organizacional** que busca unir desarrollo (Dev) y operaciones (Ops) para entregar valor de forma continua y confiable.

¿Qué es DevOps?

DevOps es el conjunto de prácticas que combina el desarrollo de software (Dev) y las operaciones de TI (Ops) para acortar el ciclo de vida del desarrollo de sistemas y proporcionar entrega continua con alta calidad.

El símbolo del infinity



DevOps es un ciclo continuo, no una línea recta.

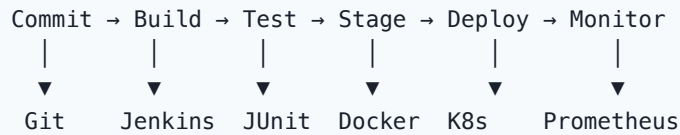
CALMS: Los 5 pilares de DevOps

Pilar	Descripción	Prácticas
Culture	Cultura de colaboración	Blameless postmortems, shared ownership
Automation	Automatización completa	CI/CD, IaC, testing automatizado
Lean	Pensamiento lean	Eliminar desperdicio, flujo continuo
Measurement	Medición continua	Métricas, monitoring, alertas
Sharing	Conocimiento compartido	Documentación abierta, blameless culture

Culture — La base de todo

- **Blameless postmortems:** analizar qué salió mal, no quién falló
- **Shared ownership:** todos son responsables de producción
- **Psychological safety:** poder reportar errores sin miedo
- **Cross-functional teams:** equipos con todas las habilidades necesarias

Automation — Eliminar trabajo manual



Qué automatizar:

- Build y compilación
- Testing (unit, integration, e2e)
- Deployment (blue-green, canary)
- Infraestructura (Terraform, Ansible)
- Monitoring y alertas

Lean — Flujo continuo

Relacionado con Lean Software Development:

- **Eliminar desperdicio**: código innecesario, wait times
- **Amplificar aprendizaje**: feedback rápido
- **Decidir lo más tarde posible**: opciones, no compromisos
- **Entregar lo más rápido posible**: batch size pequeño

Measurement — Lo que no se mide, no se mejora

Métrica	Descripción
Lead Time	Tiempo desde commit hasta producción
Deployment Frequency	Frecuencia de despliegues
Change Failure Rate	% de cambios que causan fallo
MTTR	Mean Time To Recovery
DORA Metrics	Las 4 métricas anteriores combinadas

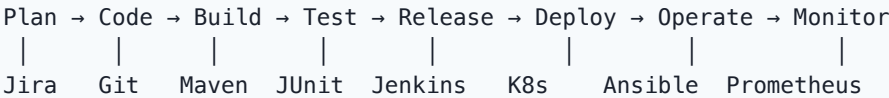
Sharing — Romper silos

- Documentación viva (no specs muertas)
- Pair programming y mob programming
- Tech talks internas
- Runbooks compartidos
- Chatops para incidentes

DevOps vs IT Operations tradicional

Aspecto	IT Ops tradicional	DevOps
Velocidad	Mensual, trimestral	Diaria, continua
Riesgo	Minimizar cambios	Cambios pequeños y frecuentes
Roles	Silos separados	Equipos cross-functionales
Herramientas	Manuales	Automatizadas
Incidentes	finger-pointing	blameless postmortems

El DevOps Toolchain



DevOps en la Software Factory

La Software Factory aplica DevOps como:

1. **Pipeline estándar**: todos los proyectos usan el mismo flujo
2. **Herramientas compartidas**: centro de plataforma común
3. **Métricas unificadas**: dashboards consolidados
4. **Gobernanza automatizada**: quality gates en pipelines
5. **Onboarding acelerado**: playbooks de DevOps

DevOps y Shift-Left

DevOps promueve mover la calidad hacia atrás:

- **Testing** en desarrollo, no en QA
- **Seguridad** en diseño, no en auditoría
- **Monitoring** desde el primer día
- **Documentation** junto al código

Relación con otros conceptos

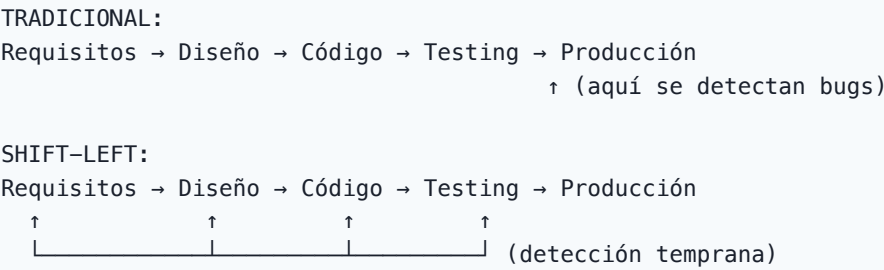
- Agile provee el mindset
- Lean provee la eficiencia
- Principios SOLID proveen calidad de código
- Shift-Left provee prevención
- Plataformas proveen self-service

Shift Left

Shift-Left: Mover Calidad Hacia Atrás

Shift-Left es la práctica de mover actividades de testing, seguridad y validación hacia las fases tempranas del ciclo de desarrollo. Cuanto antes detectes un problema, menos cuesta corregirlo.

El concepto



El costo del cambio

Fase donde se detecta	Costo relativo
Requisitos	1x
Diseño	5x
Código	10x
Testing	20x
Producción	100x

Un bug encontrado en producción cuesta **100 veces más** que uno encontrado en requisitos.

Tipos de Shift-Left

1. Testing Shift-Left

Mover el testing hacia las fases iniciales:

- **Unit Testing:** TDD (Test-Driven Development)
- **Integration Testing:** desde el primer commit
- **Code Review:** automatizado y manual
- **Static Analysis:** SonarQube, ESLint, etc.
- **Contract Testing:** validación de APIs

Prácticas:

- Escribir tests **antes** del código (TDD)
- Ejecutar tests en **cada commit** (CI)
- Usar **feature flags** para testing en producción
- Implementar **canary releases**

2. Security Shift-Left (DevSecOps)

Integrar seguridad desde el inicio:

Herramienta	Fase	Tipo
SAST	Código estático	Análisis estático
DAST	Runtime	Análisis dinámico
SCA	Dependencias	Análisis de composición
IaC Scanning	Infraestructura	Seguridad de configuración
Secret Scanning	Repositorios	Detección de secrets

Prácticas:

- Security champions en cada equipo
- Threat modeling en diseño
- Dependabot/Renovate para dependencias
- Pre-commit hooks para secrets

3. Infrastructure Shift-Left

Mover infraestructura al desarrollo:

- **Infrastructure as Code** (IaC): Terraform, Pulumi
- **Local environments**: Docker, Kind
- **Policy as Code**: OPA, Kyverno
- **Environment parity**: mismo stack en todos los ambientes

4. Observability Shift-Left

Monitoreo desde el diseño:

- **Structured logging** desde el primer día
- **Distributed tracing** en desarrollo
- **Synthetic monitoring** en CI
- **Chaos engineering** controlado

Implementación de Shift-Left

Paso 1: Evaluar el estado actual

¿Qué testing automatizado tienes?
¿Cuándo ejecutas security scans?
¿Los ambientes son consistentes?
¿El feedback loop es rápido?

Paso 2: Establecer baseline

- Definir métricas actuales
- Identificar gaps críticos
- Priorizar por impacto

Paso 3: Implementar incrementalmente

1. CI pipeline con tests básicos
2. SAST en el pipeline
3. Code review automatizado
4. Testing en desarrollo local
5. Chaos engineering controlado

Paso 4: Medir y mejorar

- Métricas de lead time
- Reduction en escape rate
- Tiempo de feedback
- Satisfacción del equipo

Beneficios

- **Reducción de costos:** bugs tempranos son más baratos
- **Mayor calidad:** prevención > detección
- **Feedback rápido:** ciclos más cortos
- **Menos estrés:** menos firefights en producción
- **Mejor seguridad:** vulnerabilities detectadas temprano

Relación con DevOps

DevOps promueve Shift-Left como parte de su filosofía:

- **Automation:** automatizar la detección
- **Measurement:** medir la efectividad
- **Sharing:** compartir conocimiento de seguridad
- **Culture:** blameless, aprendizaje continuo

Relación con Technical Debt

Technical Debt se reduce con Shift-Left porque:

- Los bugs se detectan antes de convertirse en deuda
- La deuda deliberada se gestiona con métricas
- La calidad se mantiene desde el inicio
- Los refactoring son más baratos

Shift-Left en la era de la IA

La IA potencia Shift-Left:

- **AI-powered code review**: detección automática de issues
- **Auto-generated tests**: tests generados por IA
- **Intelligent SAST**: análisis más preciso
- **Predictive analytics**: predicción de bugs

Relación con otros conceptos

- Fundamenta calidad en la Software Factory
- Se integra con prácticas de desarrollo
- Soporta métricas de calidad
- Complementa Agile con rigor técnico

Technical Debt

Technical Debt: Deuda Técnica

La **deuda técnica** es el costo implícito de retrasar mejoras técnicas. Al igual que la deuda financiera, la deuda técnica acumula "intereses" que ralentizan el desarrollo futuro.

Definición

La deuda técnica es la diferencia entre la implementación actual y la implementación óptima, considerando todos los factores de calidad.

En palabras simples: es el "atajo" que tomas hoy y que mañana te costará más tiempo resolver.

Cuadrante de Martin Fowler

Martin Fowler clasificó la deuda técnica en un modelo de 2x2:

	Deliberate	Inadvertious
Reckless	"No hay tiempo para diseñar"	"¿Qué es un patrón de diseño?"
Prudent	"Debemos lanzar ahora y refactorizar después"	"Ahora sé cómo debería ser"

Tipos de deuda

Tipo	Descripción	Ejemplo
Deliberate / Reckless	Decisión consciente pero mala	"No escribimos tests para entregar rápido"
Deliberate / Prudent	Decisión consciente y razonable	"Lanzamos MVP, refactorizamos después"
Inadvertent / Reckless	No sabías que estabas creando deuda	Copiar código sin entender patrón
Inadvertent / Prudent	Aprendizaje natural	"Ahora sé que debería haber sido diferente"

Fuentes de deuda técnica

Deuda deliberada

- Time pressure:** "Necesitamos entregar para la fecha"
- Resource constraints:** "No tenemos suficientes desarrolladores"
- Business decisions:** "El mercado no espera"

- **Technical shortcuts:** "Usamos un hack para funcionar"

Deuda inadvertida

- **Lack of knowledge:** "No sabíamos el patrón correcto"
- **Evolving requirements:** "El negocio cambió de dirección"
- **Technology evolution:** "La tecnología quedó obsoleta"
- **Team turnover:** "El conocimiento se perdió"

Métricas de deuda técnica

Métrica	Descripción
Debt Ratio	Deuda total / esfuerzo total
Code Smells	Número de code smells detectados
Technical Debt Items	Tickets de deuda backlog
Time Spent on Debt	Tiempo dedicado a pagar deuda
Interest Rate	Costo acumulado por no resolver

Estrategias de gestión

1. Medir y priorizar

Deuda técnica:

- └─ Crítica (bloquea features): Resolver AHORA
- └─ Alta (reduce velocity significativamente): Planificar este sprint
- └─ Media (causa fricción): Backlog priorizado
- └─ Baja (cosmética): Cuando haya tiempo

2. Presupuesto de deuda

- Asignar **20% del tiempo** a reducción de deuda
- Incluir deuda en **sprint planning**
- Crear **tech debt backlog** visible
- Medir **velocity impact** de la deuda

3. Prevención

- Shift-Left: detectar temprano
- Principios SOLID: diseño correcto
- **Code review:** detectar atajos antes de merge
- **Definition of Done:** incluir calidad
- **Automated testing:** prevenir regressions

4. Pago incremental

- **Refactorización continua:** no grandes migraciones
- **Boy Scout Rule:** dejar el código mejor de como lo encontraste
- **Tech debt sprints:** sprints dedicados a deuda
- **Strangler Fig Pattern:** reemplazar gradualmente

Deuda técnica y productividad

Velocity sin deuda:		100%
Velocity con 20% deuda:		75%
Velocity con 50% deuda:		50%
Velocity con 80% deuda:		25%

La deuda técnica es **exponencial**: no afecta linealmente, sino que se compone.

Deuda técnica y IA

La IA puede ayudar a gestionar deuda:

- **Detección automática:** AI para encontrar code smells
- **Refactoring asistido:** sugerencias de mejora
- **Code review inteligente:** detección de patrones problemáticos
- **Pero:** la IA también puede **crear deuda** si se usa sin criterio

Anti-patrones comunes

1. **"Refactorizaremos después":** nunca llega el "después"
2. **"Es solo un hack":** se convierte en sistema
3. **"No tenemos tiempo para tests":** el tiempo se multiplica
4. **"Funciona, no lo toques":** funciona hasta que no funciona
5. **"La deuda es inevitable":** sí, pero se puede gestionar

Relación con otros conceptos

- Shift-Left previene deuda
- Principios SOLID reducen deuda
- Lean elimina desperdicio (incluyendo deuda)
- Métricas miden deuda
- Software Factory gestiona deuda como portfolio

Conway Ley

Ley de Conway y Diseño de Sistemas

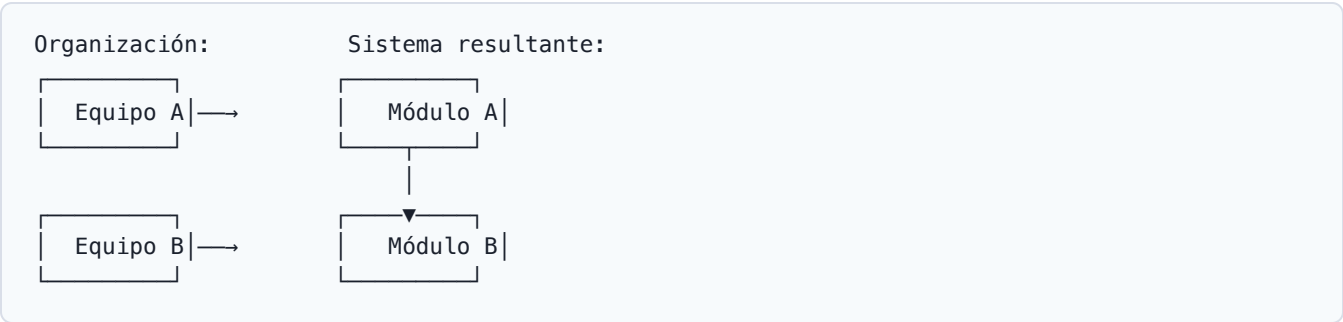
La **Ley de Conway** (1967) establece una relación profunda entre la estructura organizacional y la arquitectura de los sistemas que construye.

Definición original

"Organizaciones que diseñan sistemas están limitadas a producir diseños que son copias de las estructuras de comunicación de las organizaciones." — Melvin Conway

En otras palabras: la forma en que tu equipo se comunica determina la forma en que tu software se estructura.

Ejemplo clásico



Si los equipos no se comunican, los módulos no se comunican. Si los equipos se comunican mucho, los módulos se acoplan.

Inverse Conway Maneuver

La **inversión** de la Ley de Conway sugiere:

Diseña primero la arquitectura deseada del sistema, y luego organiza los equipos para que coincidan.

- INVERSO:
1. Diseñar arquitectura target
 2. Definir boundaries de sistemas
 3. Organizar equipos alrededor de esos boundaries
 4. Los equipos construyen lo que les corresponde

Ejemplo práctico

Arquitectura deseada	Equipo necesario
API Gateway	Equipo de Platform

Arquitectura deseada	Equipo necesario
User Service	Equipo de Identity
Payment Service	Equipo de Payments
Notification Service	Equipo de Comms

Team Topologies

Matthew Skelton y Manuel Pais formalizaron este concepto en **Team Topologies** (2019):

4 tipos de equipos

Tipo	Responsabilidad	Ejemplo
Stream-aligned	aligned con flujo de negocio	Equipo de producto
Platform	provee servicios internos	Equipo de plataforma
Enabling	habilita a otros equipos	Equipo de DevOps
Complicated-subsystem	especialización profunda	Equipo de ML/AI

3 tipos de interacción

Tipo	Descripción	Duración
Collaboration	Trabajar juntos en algo nuevo	Temporal
X-as-a-Service	Un equipo consume servicio de otro	Permanente
Facilitating	Un equipo ayuda a otro a mejorar	Temporal

Aplicación en Software Factory

La Software Factory aplica la Ley de Conway:

Estructura tipo

PLATFORM TEAM (Infra, CI/CD, Observability)		
PRODUCT TEAM A (User)	PRODUCT TEAM B (Pay)	PRODUCT TEAM C (Notif)

Principios

1. **Equipos cross-functionales**: cada equipo tiene todas las habilidades
2. **Ownership claro**: cada equipo es dueño de su dominio
3. **Interfaces definidas**: contratos entre equipos
4. **Platform como servicio**: equipo central soporta a otros
5. **Reducir dependencias**: minimizar sincronización entre equipos

Errores comunes

1. Microservicios con equipo monolítico

- ✗ Un equipo construye 50 microservicios
- ✓ Cada equipo construye 2-3 microservicios relacionados

2. Monolito con equipos aislados

- ✗ Un monolito donde cada equipo toca todo
- ✓ Monolito con módulos claros por equipo

3. Equipos sin boundaries claros

- ✗ "El equipo de backend hace todo"
- ✓ "Equipo A: User Domain, Equipo B: Payment Domain"

Ley de Conway y Arquitectura Limpia

Arquitectura Limpia se beneficia de la Ley de Conway:

- **Dependency Rule**: las dependencias van hacia adentro → equipos independientes
- **Ports & Adapters**: interfaces claras → contratos entre equipos
- **Use Cases**: lógica de negocio separada → ownership por dominio

Métricas de Conway

Métrica	Descripción
Communication patterns	Cómo se comunican los equipos
Dependency graph	Dependencias entre módulos
Code ownership	Quién es dueño de qué
Change coupling	Qué cambia junto

Relación con otros conceptos

- Fundamenta la estructura de la Software Factory

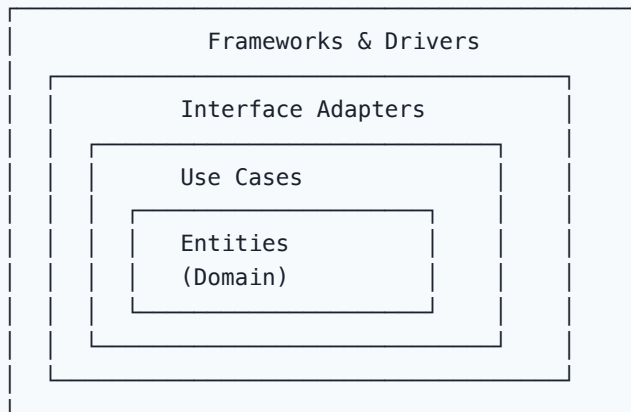
- Se integra con Arquitectura Limpia
- Se soporta en Plataformas de ingeniería
- Complementa segmentación de software
- Aplicable en procesos y flujos

Arquitectura Limpia

Arquitectura Limpia y Principios SOLID

La **Arquitectura Limpia** (Clean Architecture), formalizada por Robert C. Martin, es un patrón de arquitectura de software que separa concerns en capas concéntricas, donde las dependencias apuntan siempre hacia adentro.

El modelo de capas



Las 4 capas

1. Entities (Domain)

El núcleo del sistema. Contiene las **entidades de negocio** y **reglas de negocio**.

- **Independiente** de cualquier framework
- **No tiene** dependencias externas
- **Contiene** la lógica de negocio pura
- **Cambia** solo por cambios en el negocio

```
# Ejemplo de entity
class Order:
    def __init__(self, items: list[Item]):
        self.items = items
        self.status = "pending"

    def calculate_total(self) -> float:
        return sum(item.price * item.quantity for item in self.items)

    def approve(self):
        if self.status != "pending":
```

```
        raise InvalidOrderStatus()
    self.status = "approved"
```

2. Use Cases

Orquesta las entidades para realizar **operaciones específicas** del sistema.

- **Define** las operaciones disponibles
- **Usa** entidades y reglas de negocio
- **No sabe** de UI ni de persistencia
- **Cambia** por cambios en requisitos

```
class CreateOrderUseCase:
    def __init__(self, order_repository: OrderRepository):
        self.order_repository = order_repository

    def execute(self, items: list[Item]) -> Order:
        order = Order(items)
        order.approve()
        self.order_repository.save(order)
        return order
```

3. Interface Adapters

Convierte datos entre las capas. Incluye:

- **Controllers**: reciben input del mundo externo
- **Presenters**: preparan output para la UI
- **Gateways**: adaptan interfaces de persistencia
- **Mappers**: transforman datos entre formatos

```
class OrderController:
    def __init__(self, create_order: CreateOrderUseCase):
        self.create_order = create_order

    def create(self, request: CreateOrderRequest) -> OrderResponse:
        order = self.create_order.execute(request.items)
        return OrderResponse(order.id, order.status)
```

4. Frameworks & Drivers

El shell más externo. Contiene:

- **UI**: web, mobile, CLI
- **Frameworks**: Django, Spring, Express
- **Drivers**: databases, external APIs
- **Config**: settings, environment variables

La Dependency Rule

Las dependencias de código deben apuntar solo hacia adentro. Nada en una capa más interna debe saber algo de las capas más externas.

Entidades ← Use Cases ← Interface Adapters ← Frameworks

↑ ↑ ↑ ↑

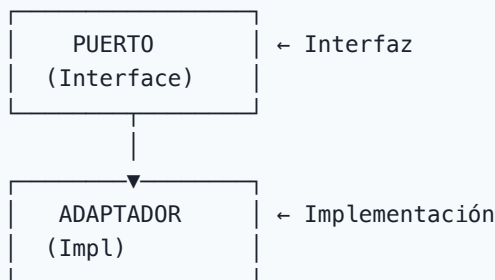
(nada depende de nada afuera)

En la práctica:

- Entidades no importan Use Cases
- Use Cases no importan Controllers
- Controllers no importan UI
- UI no importa databases directamente

Ports & Adapters (Hexagonal Architecture)

Variante de la Clean Architecture donde se definen **puertos** (interfaces) y **adaptadores** (implementaciones):



Ejemplo:

- **Puerto:** OrderRepository (interface)
- **Adaptador PostgreSQL:** PostgresOrderRepository
- **Adaptador MongoDB:** MongoOrderRepository

Permite cambiar implementaciones sin cambiar la lógica de negocio.

SOLID aplicado a arquitectura

Los principios SOLID se aplican a nivel de arquitectura:

Principio	Aplicación arquitectónica
SRP	Cada capa tiene una responsabilidad
OCP	Nuevas implementaciones sin modificar código existente
LSP	Adaptadores intercambiables

Principio	Aplicación arquitectónica
ISP	Interfaces de puertos pequeñas y específicas
DIP	Dependencias apuntan hacia adentro

Beneficios

- **Testabilidad:** cada capa es testeable independientemente
- **Flexibilidad:** cambiar UI, DB o framework sin afectar negocio
- **Maintainability:** cambios localizados en una capa
- **Scalability:** escalar capas independientemente
- **Team autonomy:** equipos pueden trabajar en diferentes capas

Cuando NO usar Clean Architecture

- **Prototipos rápidos:** over-engineering
- **Scripts simples:** no justifica la complejidad
- **CRUD básico:** puede ser demasiado
- **Equipos pequeños:** puede ser overhead

Relación con otros conceptos

- Se fundamenta en principios SOLID
- Se aplica en monolito y microservicios
- Se soporta en plataformas
- Se beneficia de Ley de Conway
- Se integra con Software Factory

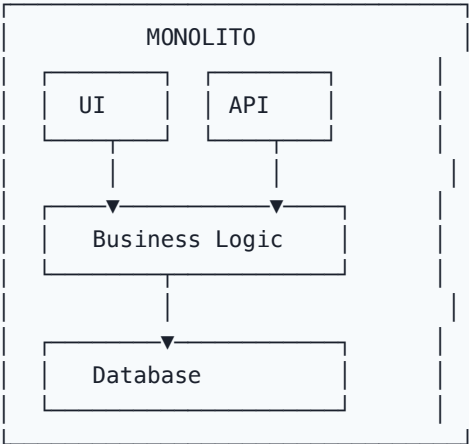
Monolito Microservicios

De Monolito a Microservicios

La evolución de monolitos a microservicios es una de las decisiones arquitectónicas más importantes (y costosas) que puede tomar una organización.

¿Qué es un Monolito?

Un **monolito** es una aplicación donde toda la lógica de negocio, UI y persistencia están acopladas en un solo deployable.



Pros del Monolito

Ventaja	Descripción
Simplicidad	Un solo deploy, una base de código
Performance	Comunicación en memoria, sin network
Debugging	Stack trace completo, una sola app
Consistencia transaccional	ACID straightforward
Desarrollo rápido	No hay overhead de distribución

Contras del Monolito

Desventaja	Descripción
Escalabilidad	Todo o nada, no por componente
Acoplamiento	Un cambio puede romper todo

Desventaja	Descripción
Deploy risk	Desplegar todo junto, alto riesgo
Technology lock-in	Toda la app usa el mismo stack
Team conflicts	Múltiples equipos en misma codebase

¿Cuándo dividir en microservicios?

Señales de que estás listo

1. **Equipos estancados**: demasiados developes en una codebase
2. **Deployments dolorosos**: desplegar toma horas
3. **Escalabilidad asimétrica**: necesitas escalar solo un componente
4. **Resiliencia**: un componente falla y cae todo
5. **Bounded contexts claros**: los dominios de negocio están bien definidos

Señales de que NO estás listo

1. **Equipo pequeño** (< 10 desarrolladores)
2. **Dominio no claro** (no sabes qué es cada bounded context)
3. **Sin DevOps maduro** (no tienes CI/CD)
4. **Sin monitoring** (no sabes qué está pasando)
5. **Presión por features** (no puedes invertir en arquitectura)

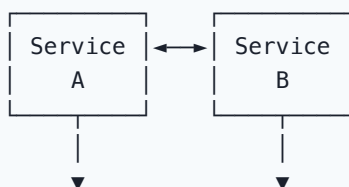
Patrones de microservicios

1. API Gateway

```
Client → API Gateway → Service A
                        → Service B
                        → Service C
```

- Punto de entrada único
- Routing, authentication, rate limiting
- Ejemplos: Kong, AWS API Gateway, Envoy

2. Service Mesh




```
Service Mesh
(Istio, Linkerd, etc.)
```

Infraestructura de red para microservicios:

- **mTLS**: seguridad entre servicios
- **Load balancing**: distribución de tráfico
- **Observability**: tracing, metrics, logging
- **Circuit breaking**: resiliencia

3. Event-Driven Architecture

```
graph LR
    A[Service A] --event--> B[Event Bus]
    B --event--> C[Service B]
    B --event--> D[Service C]
```

- **Kafka**, RabbitMQ, NATS
- **Loose coupling**: servicios no se conocen
- **Eventual consistency**: consistency saga pattern

4. Saga Pattern

Para transacciones distribuidas:

```
graph LR
    A[Create Order] --> B[Reserve Inventory]
    B --> C[Process Payment]
    C --> D[Confirm Order]
    B -- compensating transactions --> A
    C -- compensating transactions --> B
```

Estrategia de migración: Strangler Fig Pattern

No se puede migrar de golpe. El **Strangler Fig** reemplaza gradualmente:

1. Identificar bounded context
2. Crear nuevo microservicio
3. Router redirige tráfico gradualmente
4. Migrar datos
5. Retirar módulo del monolito

Pasos prácticos

1. **Identificar**: qué bounded context migrar primero
2. **Aislar**: crear interfaz clara (API)
3. **Extraer**: mover lógica a nuevo servicio
4. **Redirect**: router apunta al nuevo servicio
5. **Validate**: verificar comportamiento

6. **Retire**: eliminar del monolito

Consideraciones operacionales

Aspecto	Monolito	Microservicios
Deployment	Simple, un solo artefacto	Complejo, orquestación
Monitoring	Una app, un dashboard	Múltiples servicios
Debugging	Stack trace completo	Distributed tracing
Testing	Integration tests simples	Contract tests, e2e
Data	Una DB, transacciones ACID	DB por servicio, eventual consistency

Relación con otros conceptos

- Se beneficia de Arquitectura Limpia
- Se soporta en Ley de Conway
- Requiere DevOps maduro
- Se mide con métricas DORA
- Se gestiona con plataformas

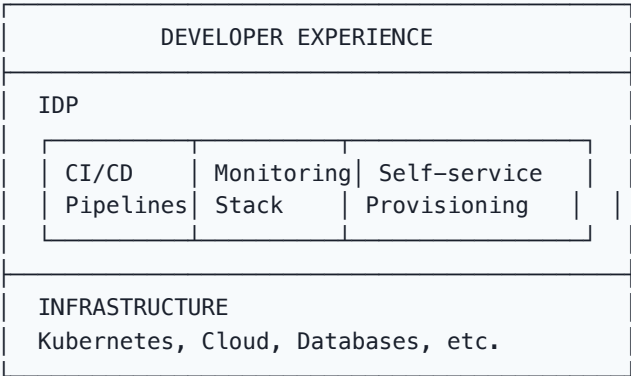
Plataformas Ingenieria

Plataformas de Ingeniería

Una **plataformas de ingeniería** es un conjunto de herramientas, procesos y documentación que proporciona a los desarrolladores una experiencia unificada para construir, desplegar y operar software.

Internal Developer Platform (IDP)

Una **IDP** es la capa de abstracción que simplifica la infraestructura para los desarrolladores.



Componentes típicos de una IDP

Componente	Herramientas	Función
CI/CD	GitHub Actions, GitLab CI	Build y deploy automatizado
Container Orchestration	Kubernetes, ECS	Ejecución de servicios
Service Catalog	Backstage	Inventario de servicios
Secrets Management	Vault, AWS SM	Gestión de secretos
Observability	Grafana, Datadog	Monitoring y alertas
Infrastructure as Code	Terraform, Pulumi	Provisioning
Developer Portal	Backstage, Port	Self-service

Golden Paths

Los **Golden Paths** son caminos predefinidos y validados que facilitan la creación de nuevos servicios o features.

¿Qué incluye un Golden Path?

```
golden_path:
  name: "Nuevo microservicio"
  template:
    - repo_scaffolding: "service-template"
    - ci_pipeline: "standard-pipeline"
    - cd_config: "k8s-standard"
    - monitoring: "grafana-dashboard"
    - alerts: "pagerduty-standard"
    - docs: "README-template"
  stages:
    - name: "Development"
      tools: ["local-dev", "pre-commit"]
    - name: "CI"
      tools: ["github-actions", "sonarqube"]
    - name: "CD"
      tools: ["argocd", "terraform"]
    - name: "Operations"
      tools: ["grafana", "pagerduty"]
```

Beneficios de los Golden Paths

1. **Velocidad:** nuevo servicio en minutos, no días
2. **Consistencia:** todos los servicios siguen el mismo patrón
3. **Calidad:** las mejores prácticas están embebidas
4. **Onboarding:** nuevos desarrolladores productivos rápido
5. **Gobernanza:** compliance automático

Backstage — El portal del desarrollador

Backstage es un portal de desarrolladores open-source creado por Spotify.

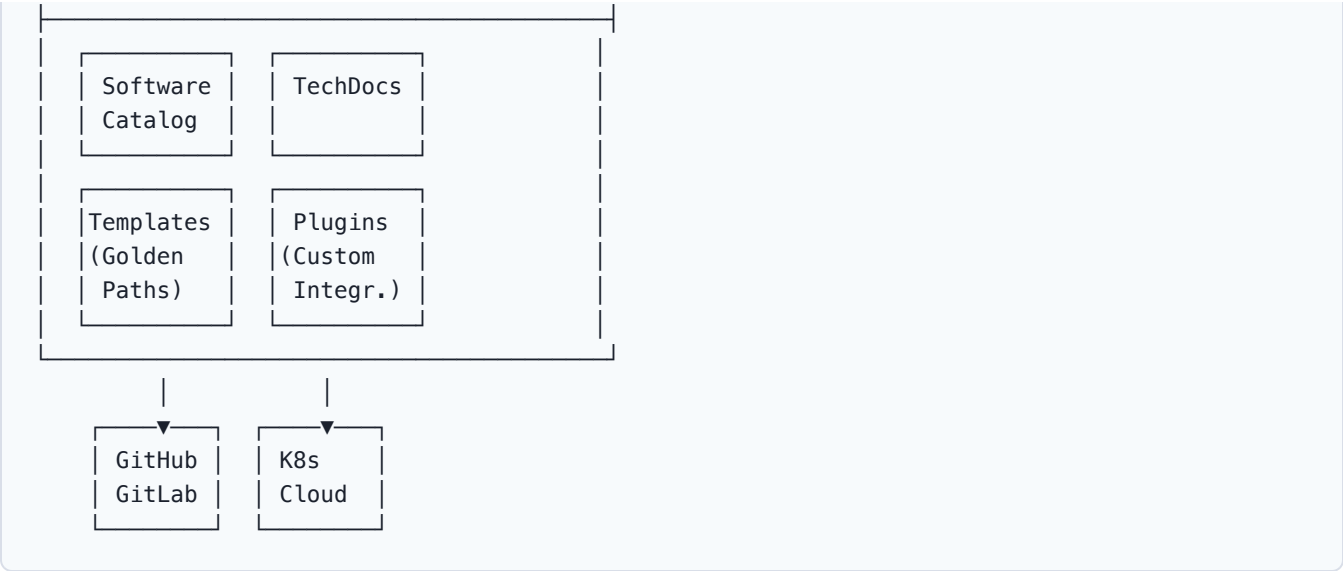
Funcionalidades principales

Feature	Descripción
Service Catalog	Inventario de todos los servicios
Software Templates	Golden paths como templates
TechDocs	Documentación como código
Plugin System	Extensible con plugins
API Documentation	APIs documentadas automáticamente

Arquitectura de Backstage



BACKSTAGE



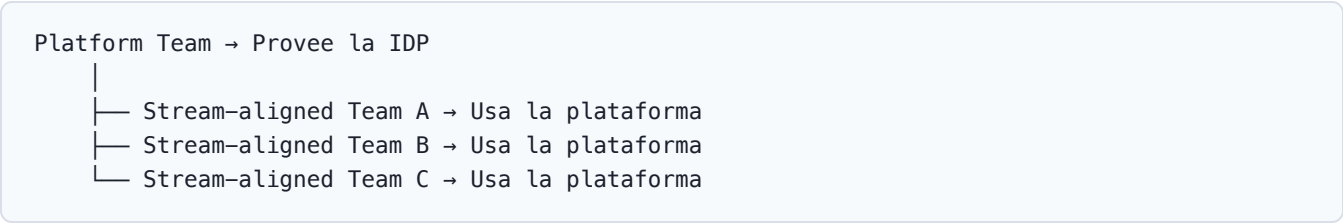
Self-Service para desarrolladores

Una buena plataforma permite a los desarrolladores:

Acción	Sin plataforma	Con plataforma
Crear servicio	Ticket + 2 semanas	Template + 10 minutos
Deployar	Manual + approval	CI/CD automático
Monitorear	Configurar manual	Dashboard automático
Acceder a DB	Ticket + 1 semana	Self-service con policy
Rotar secretos	Ops ticket	Vault UI

El modelo Team Topologies + Platform

Aplicando Team Topologies:



Responsabilidades del Platform Team:

- Mantener la infraestructura base
- Proveer Golden Paths
- Gestionar CI/CD shared
- Soporte a equipos de producto

Métricas de plataforma

Métrica	Descripción
Time to First Deploy	Tiempo para primer deploy de nuevo servicio
Developer Satisfaction	Encuesta de satisfacción
Platform Adoption	% de servicios en la plataforma
Self-service Ratio	% de operaciones sin ticket a ops
Deployment Frequency	Frecuencia de deploys

Plataformas en la era de la IA

La IA potencia las plataformas:

- **AI-powered scaffolding**: generación automática de código
- **Intelligent monitoring**: anomaly detection
- **Automated onboarding**: AI guides para nuevos desarrolladores
- **Smart Golden Paths**: sugerencias basadas en patrones

Relación con otros conceptos

- Implementa la Software Factory
- Se beneficia de Ley de Conway
- Soporta DevOps
- Se integra con microservicios
- Se mide con métricas de productividad

Software Categoría A

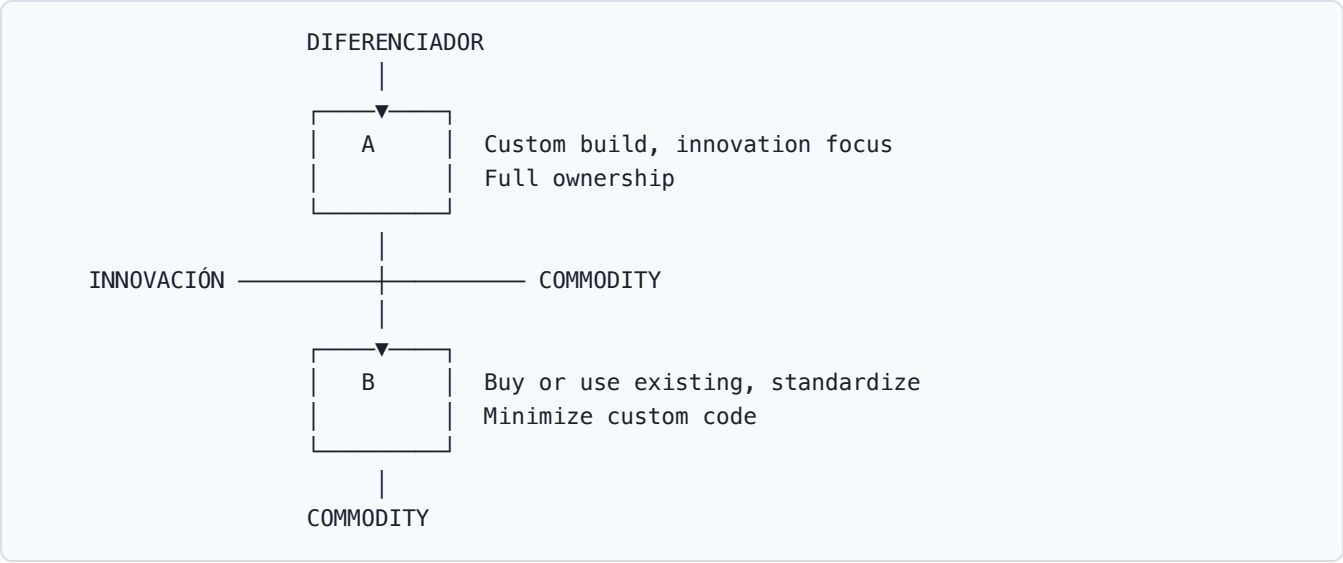
Software de Categoría A vs B

No todo software tiene el mismo valor estratégico. Distinguir entre software **diferenciador** (Categoría A) y software **commodity** (Categoría B) es clave para tomar decisiones arquitectónicas y organizacionales correctas.

Definiciones

Categoría	Descripción	Ejemplo
Categoría A	Diferenciador de negocio, genera ventaja competitiva	Motor de recomendación, algoritmo de pricing
Categoría B	Commodity, necesario pero no diferenciador	Auth, logging, email sending, CMS

El framework de Martin Fowler



Características de cada categoría

Software de Categoría A (Diferenciador)

Aspecto	Enfoque
Estrategia	Build custom, innovar
Equipo	Senior engineers, domain experts
Arquitectura	Diseñada para cambios frecuentes
Testing	Exhaustivo, e2e, performance

Aspecto	Enfoque
Deployment	Continuous, feature flags
Métricas	Business impact, revenue
I+D	Invertir en innovación

Software de Categoría B (Commodity)

Aspecto	Enfoque
Estrategia	Buy, SaaS, open-source
Equipo	Standard skills, ops focus
Arquitectura	Simple, estable, estándar
Testing	Básico, smoke tests
Deployment	Manual o semi-automático
Métricas	Uptime, cost
I+D	Mínimo, mantener

Ejemplos prácticos

Categoría A (Diferenciador)

- **Algoritmo de recomendación:** core del negocio
- **Motor de pricing dinámico:** ventaja competitiva
- **ML model:** predicciones específicas del dominio
- **UX diferenciador:** experiencia única
- **Integraciones estratégicas:** partnerships clave

Categoría B (Commodity)

- **Authentication:** Auth0, Firebase Auth, Keycloak
- **Email:** SendGrid, SES, Mailgun
- **Logging:** ELK, Datadog, Grafana
- **CMS:** WordPress, Contentful, Strapi
- **Payment processing:** Stripe, PayPal (el procesamiento, no la integración)
- **CI/CD:** GitHub Actions, GitLab CI

Implicaciones para la organización

Estructura de equipos

Categoría A	Categoría B
Equipo dedicado, cross-functional	Equipo compartido, rotations
Senior engineers	Mid-level + automation
Autonomous	Centralized
Close to business	Close to ops

Arquitectura

Categoría A	Categoría B
Microservicios si aporta	Monolito o servicios estándar
Custom build	SaaS o open-source
Event-driven	Request-response
Polyglot	Estándar

Presupuesto

Categoría A: 70% del presupuesto de desarrollo
Categoría B: 30% del presupuesto (o menos)

La trampa de tratar B como A

Muchas organizaciones construyen **custom** software de Categoría B:

- Custom auth system (cuando existe Auth0)
- Custom CMS (cuando existe Contentful)
- Custom CI/CD (cuando existen GitHub Actions)

Resultado:

- Alto costo de mantenimiento
- Features retrasadas
- Equipo sobrecargado
- Competencia desventajosa

La trampa de tratar A como B

Organizaciones que **compran** o **externalizan** software diferenciador:

- Usar un CMS para el core del negocio
- Contratar consultora para el algoritmo principal
- Usar SaaS para la ventaja competitiva

Resultado:

- Pérdida de diferenciación
- Dependencia del vendor
- Imposibilidad de innovar

Relación con la Software Factory

La Software Factory aplica esta clasificación:

1. **Portfolio management**: categorizar cada sistema
2. **Resource allocation**: invertir más en A
3. **Architecture decisions**: basadas en categoría
4. **Build vs Buy framework**: decisión informada
5. **Team structure**: equipos apropiados para cada categoría

Decisiones de Build vs Buy

Pregunta	Si es Categoría A	Si es Categoría B
¿Diferencia el negocio?	Build	Buy
¿Es core del producto?	Build	Buy
¿Hay alternativas maduras?	Evaluar	Buy
¿Necesitamos customización extrema?	Build	Reconsiderar
¿Tenemos expertise interno?	Build	Evaluar

Relación con otros conceptos

- Informa producto vs proyecto
- Se aplica en decisiones arquitectónicas
- Se soporta en plataformas
- Se mide con métricas de negocio
- Se integra con estructura de equipos

Producto Vs Proyecto

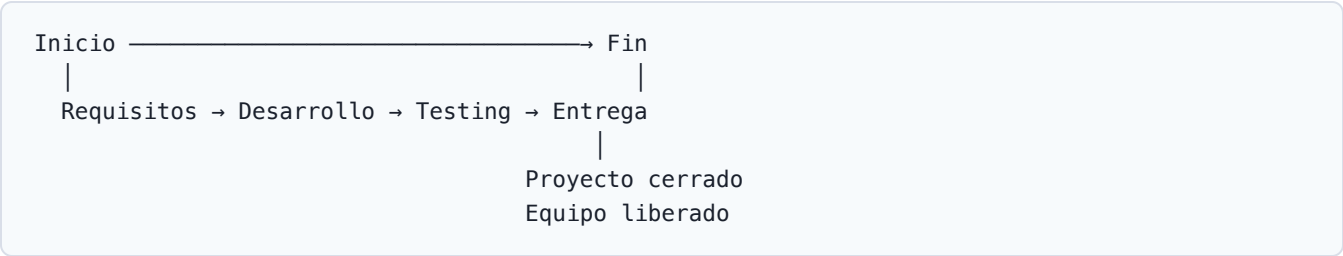
Modelo Producto vs Proyecto

La distinción entre **pensamiento de producto** y **pensamiento de proyecto** es fundamental para la forma en que se construye y mantiene software a largo plazo.

Definiciones

Aspecto	Modelo Proyecto	Modelo Producto
Enfoque	Entregar un output	Crear y evolucionar un valor
Fin	Tiene fecha de cierre	Continuo, mientras aporte valor
Éxito	Entregar a tiempo/presupuesto	Impacto en negocio y usuarios
Equipo	Se disuelve al final	Permanente, cross-functional
Ownership	Temporal	Permanente
Feedback	Al final del proyecto	Continuo

El modelo Proyecto



Características

- **Presupuesto fijo**: se aprueba al inicio
- **Alcance definido**: se fija en requisitos
- **Timeline rígida**: fechas de entrega hard
- **Equipo temporal**: se arma y se desarma
- **Definition of Done**: entregar el output

Anti-patrones del modelo proyecto

1. **"Feature factory"**: 产出 sin medir impacto
2. **"Throw it over the wall"**: desarrollo → ops sin continuidad
3. **"No es mi proyecto"**: falta de ownership post-entrega
4. **"Presupuesto agotado"**: no hay recursos para mejoras

5. "El proyecto terminó": software sin mantenimiento

El modelo Producto



Características

- **Presupuesto continuo:** invertir mientras haya ROI
- **Alcance evolutivo:** se adapta al mercado
- **Timeline flexible:** releases incrementales
- **Equipo permanente:** ownership continuo
- **Definition of Done:** impacto medido

Beneficios del modelo producto

1. **Continuidad:** el equipo conoce el dominio
2. **Aprendizaje:** feedback continua mejora el producto
3. **Responsabilidad:** el equipo es dueño del resultado
4. **Calidad:** inversiones en deuda técnica y mejoras
5. **Velocidad:** con el tiempo, el equipo es más rápido

Comparación detallada

Dimensión	Proyecto	Producto
Planning	Big bang, up front	Roadmap continuo
Métricas	On time, on budget	Revenue, retention, NPS
Riesgo	Alto (todo al final)	Bajo (feedback temprano)
Deuda técnica	Se ignora	Se gestiona activamente
Equipos	Siloed por proyecto	Cross-functional permanente
Documentación	Extensiva, obsoleta	Viva, continuously updated
Deploy	Big releases	Continuous delivery

Product Thinking en la práctica

1. Product Roadmap vs Project Plan

Project Plan:

Q1: Requisitos
Q2: Desarrollo
Q3: Testing
Q4: Lanzamiento

Product Roadmap:

Q1: Validar problema + MVP
Q2: Primeros usuarios + learning
Q3: Growth + features clave
Q4: Monetización + escalar

2. OKRs vs Task Lists

Task List (Proyecto):

- ☐ Crear formulario de registro
- ☐ Implementar backend
- ☐ Diseñar UI
- ☐ Escribir tests

OKRs (Producto):

- **Objective:** Onboarding exitoso
- KR1: 80% completion rate
- KR2: < 3 minutos promedio
- KR3: NPS > 8

3. Feature Teams vs Product Teams

Feature Team:

- Recibe specification
- Implementa
- Pasa a QA
- Se desarma

Product Team:

- Identifica problema
- Diseña solución
- Implementa

- Mide impacto
- Itiera

El modelo híbrido

En la práctica, muchas organizaciones usan ambos:

Tipo de trabajo	Modelo apropiado
Core product	Producto
Strategic initiatives	Proyecto (con scope acotado)
Maintenance	Producto (continuo)
Innovation/Exploration	Proyecto (time-boxed)
Compliance	Proyecto (deadline driven)

Relación con la Software Factory

La Software Factory moderna favorece el modelo producto:

1. **Equipos permanentes**: no se arman/desarmas por proyecto
2. **Métricas de producto**: no solo de output
3. **Ownership continuo**: el equipo es dueño del sistema
4. **Mejora continua**: invertir en calidad y features
5. **Categoría A**: el software diferenciador SIEMPRE es producto

Relación con otros conceptos

- Se fundamenta en Agile
- Se integra con Categoría A vs B
- Se soporta en plataformas
- Se mide con métricas de producto
- Complementa Lean

Lean Software

Lean Software Development

Lean Software Development es una adaptación de los principios de manufacturing Lean (Toyota Production System) al desarrollo de software. Fue formalizada por Mary y Tom Poppendieck en 2003.

Los 7 Principios de Lean

1. Eliminate Waste

→ Eliminar desperdicio
2. Amplify Learning

→ Amplificar aprendizaje
3. Decide as Late as Possible

→ Decidir lo más tarde posible
4. Deliver as Fast as Possible

→ Entregar lo más rápido posible
5. Empower the Team

→ Empoderar al equipo
6. Build Integrity In

→ Construir integridad desde adentro
7. See the Whole

→ Ver el todo

1. Eliminate Waste — Eliminar desperdicio

"Anything that does not add value to the product is waste."

Tipos de desperdicio en software

Desperdicio	Ejemplo	Solución	
Partially done work	Features a medias	WIP limits, completion before new	
Extra features	Over-engineering	YAGNI, MVP	
Relearning	Knowledge not shared	Pair programming, docs vivas	
Handoffs	Dev → QA → Ops	Cross-functional teams	
Waiting	Bloqueado por otro equipo	Autonomía, reducir dependencias	
Defects	Bugs que llegan a producción	[[../01-Fundamentos/08-shift-left	Shift-Left]]
Task switching	Multitasking	Focus, WIP limits	

Identificar desperdicio

- Preguntas para identificar desperdicio:
1. ¿Qué código escribimos pero nadie usa?

2. ¿Qué features nadie pidió?

3. ¿Qué meetings no agregan valor?

4. ¿Qué procesos son overhead puro?
5. ¿Qué esperas causan retrasos?

2. Amplify Learning — Amplificar aprendizaje

"The only way to win is to learn faster than anyone else."

Prácticas:

- **TDD**: aprender mientras se diseña
- **Pair Programming**: knowledge sharing inmediato
- **Retrospectivas**: aprendizaje del equipo
- **Code Review**: aprendizaje mutuo
- **A/B Testing**: learning de usuarios reales
- **Spike solutions**: prototipos para aprender

Feedback loops:

Code → Test → Deploy → Measure → Learn → Code

```
graph LR; Code --> Test --> Deploy --> Measure --> Learn --> Code; Learn -- feedback --> Code;
```

3. Decide as Late as Possible — Decidir lo más tarde posible

"Keep your options open as long as possible, but make the decisions at the last responsible moment."

Aplicación:

- **Architecture**: no over-designar antes de saber
- **Technology choices**: evaluar cuando sea necesario
- **Feature scope**: ajustar según aprendizaje
- **Infrastructure**: cloud para flexibilidad

Beneficios:

- Más información para decidir
- Menos assumption-driven mistakes
- Mayor flexibilidad

4. Deliver as Fast as Possible — Entregar lo más rápido posible

"The faster you deliver, the sooner you get feedback, the sooner you can improve."

Prácticas:

- **CI/CD**: delivery continua
- **Feature flags**: entregar parcialmente

- **MVP**: lo mínimo viable primero
- **Small batches**: cambios pequeños y frecuentes
- **DevOps**: automatizar todo

Velocidad ≠ prisa:

- Velocidad inteligente: entregar valor rápido
- Prisa: entregar sin calidad

5. Empower the Team — Empoderar al equipo

"Motivated individuals are the core of any successful project."

Principios:

- **Autonomía**: decidir CÓMO trabajar
- **Trust**: confiar en el equipo
- **Ownership**: el equipo es dueño del resultado
- **Psychological safety**: poder fallar sin miedo

Anti-patrones:

- Micro-management
- Decisiones top-down sin input
- Blame culture
- Silos de información

6. Build Integrity In — Construir integridad desde adentro

"Integrity is not something you add later — it's built in from the start."

Tipos de integridad:

Tipo	Descripción
Perceived integrity	El usuario siente que todo se conecta
Conceptual integrity	El diseño es coherente y consistente
Technical integrity	El código es mantenible y testeable

Prácticas:

- Principios SOLID
- Arquitectura Limpia
- Technical Debt management
- Code review y testing

7. See the Whole — Ver el todo

"Optimizing a part never optimizes the whole."

Aplicación:

- **System thinking**: ver efectos en cascada
- **Value stream mapping**: entender el flujo completo
- **Cross-functional teams**: visión holística
- **Metrics**: medir el sistema, no solo componentes

Ejemplo:

Optimizar solo coding → el bottleneck se mueve a testing
Optimizar solo testing → el bottleneck se mueve a deployment
Optimizar todo junto → mejora real del sistema

Lean y Agile

Lean y Agile se complementan:

Agile se enfoca en	Lean se enfoca en
Valores y principios	Eliminar desperdicio
Entregar software	Entregar valor
Colaboración	Eficiencia del sistema
Responder al cambio	Reducir lead time
Individuos	Flujo de trabajo

Lean y DevOps

DevOps es la implementación práctica de Lean en operaciones:

Lean Principle	DevOps Practice
Eliminate waste	CI/CD, automation
Amplify learning	Monitoring, feedback
Decide as late	Feature flags, canary
Deliver fast	Continuous deployment
Empower team	Blameless culture
Build integrity	Testing, quality gates
See the whole	Observability, DORA metrics

Implementación de Lean

Paso 1: Identificar desperdicio

- Mapear value stream actual
- Identificar wait times y overhead
- Medir lead time

Paso 2: Establecer flujo

- Implementar WIP limits
- Reducir batch sizes
- Crear pull systems

Paso 3: Medir y mejorar

- Métricas de lead time
- Reducir tiempo de ciclo
- Aprendizaje continuo

Relación con otros conceptos

- Fundamenta DevOps y su filosofía
- Se integra con Agile
- Complementa gestión de deuda técnica
- Se aplica en Software Factory
- Se mide con métricas de flujo

README

Fundamentos

Conceptos esenciales que sostienen el modelo de Software Factory y la ingeniería de software moderna.

Contenido

#	Archivo	Tema	Nivel	
01	[[../01-Fundamentos/01-que-es-software-factory	¿Qué es una Software Factory?]]	Definición, características y componentes	Principiante
02	[[../01-Fundamentos/02-evolucion-software-factory	Evolución de la Software Factory]]	De talleres a fábricas de software	Intermedio
03	[[../01-Fundamentos/03-era-ia-contexto	La Era de la IA]]	Contexto y cambio de paradigma	Principiante
04	[[../01-Fundamentos/04-principios-ingenieria-software	Principios de Ingeniería]]	SOLID, DRY, KISS, YAGNI, SoC, LoD	Intermedio
05	[[../01-Fundamentos/05-modelos-madurez	Modelos de Madurez]]	CMMI, SPICE, ISO/IEC 33000	Avanzado
06	[[../01-Fundamentos/06-manifiesto-agile	El Manifiesto Ágil]]	4 valores y 12 principios	Principiante
07	[[../01-Fundamentos/07-devops-filosofia	DevOps como Filosofía]]	CALMS y cultura DevOps	Principiante
08	[[../01-Fundamentos/08-shift-left	Shift-Left]]	Mover calidad hacia atrás	Intermedio
09	[[../01-Fundamentos/09-technical-debt	Technical Debt]]	Deuda técnica y gestión	Intermedio
10	[[../01-Fundamentos/10-conway-ley	Ley de Conway]]	Organizaciones y sistemas	Intermedio
11	[[../01-Fundamentos/11-arquitectura-limpia	Arquitectura Limpia]]	Capas, regla de dependencia, puertos	Avanzado
12	[[../01-Fundamentos/12-monolito-microservicios	De Monolito a Microservicios]]	Patrones y service mesh	Avanzado
13	[[../01-Fundamentos/13-plataformas-ingenieria	Plataformas de Ingeniería]]	IDP, Golden Paths, Backstage	Intermedio
14	[[../01-Fundamentos/14-software-categoria-a	Software Categoría A vs B]]	Commodity vs innovación	Avanzado

#	Archivo	Tema	Nivel	
15	[[../01-Fundamentos/15-producto-vs-proyecto	Producto vs Proyecto]]	Product thinking vs project mindset	Intermedio
16	[[../01-Fundamentos/16-lean-software	Lean Software Development]]	Los 7 principios de Lean	Intermedio

Secciones relacionadas

- 02 — Estructura
- 03 — Procesos
- 04 — Prácticas
- 05 — Herramientas
- 06 — Métricas
- 07 — IA y Software